

IT5106: SOFTWARE DEVELOPMENT PROJECT

INTERIM REPORT

WEB-BASED VEHICLE AUCTION PLATFORM

ACADEMIC YEAR – 2024

Field	Details
Candidate Name	Kavindu Kamil Jehan Hamid
Index Number	R222226
Contact	+94 76 183 8999 kavindukamil@gmail.com
Client	Tapro Japan Co. Ltd., Tokyo, Japan
Supervisor	Dr. Subodha Hettiachchi Gunawardena, Senior Lecturer, Faculty of Engineering, University of Ruhuna
Submission Month	April 2026

Abstract

Tapro Japan Co. Ltd., a vehicle exporter based in Tokyo, Japan, currently manages its auction workflow through informal messaging channels and manual bid tracking. This approach introduces significant operational inefficiencies including missed bids, ambiguous pricing, and the inability to enforce participation by verified buyers only.

This interim report documents the design and ongoing development of a web-based vehicle auction platform intended to digitize and automate the client's entire auction process. The system enables public browsing of vehicle listings and allows verified registered buyers to place competitive bids on active auctions. A privileged administrator interface provides tools for vehicle management, user verification, auction scheduling, bid monitoring, and winning-bidder selection.

The solution is being developed as a three-tier web application using React.js for the frontend, Node.js with Express for the backend REST API, and PostgreSQL for relational data storage. JWT-based authentication enforces secure, role-separated access. A modular approach has been adopted to facilitate iterative development and future extensibility.

At the time of this report, requirements analysis and system design are complete, backend authentication and vehicle-management API endpoints have been implemented, and frontend scaffolding with routing is in place. Remaining work covers the bidding module, admin dashboard, notifications, integration testing, and final deployment. The system is on track for submission by the end of July 2026.

Table of Contents

1	INTRODUCTION	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Motivation.....	1
1.4	Objectives.....	2
2	ANALYSIS	3
2.1	Existing System.....	3
2.2	Literature Review.....	3
2.2.1	Online Auction Models	3
2.2.2	Web Application Architecture	3
2.2.3	Authentication and Role Based Access Control	4
2.2.4	Concurrent Bid Handling	4
2.2.5	Similar Commercial Systems.....	4
2.3	Functional Requirements	5
2.4	Non-Functional Requirements	6
3	Design of Solution	7
3.1	System Architecture	7
3.2	Methodology	7
3.3	Use Case Diagrams	7
3.4	Entity Relationship Diagram.....	11
3.5	Logical Database Design	11
3.6	Class Diagram Design.....	12
3.7	Sequence Diagram Design	12
3.8	Technology Selection.....	13
3.9	Current Development Status	14
3.10	Design Evolution and Earlier Approaches	15
3.10.1	Iteration 1: Initial Security and Authorization Design	15
3.10.2	Issues Identified and Design Refinements Made	15

Table of Figures

Figure 1 System High Level Architecture.....	7
Figure 2 Entity Relationship Diagram	11
Figure 3 Logical Database Design	11
Figure 4 Class Diagram.....	12
Figure 5 Sequence Diagram.....	12

List of Tables

Table 1 Functional Requirements classified by priority	5
Table 2 UC-01 – Register Account	8
Table 3 UC-02 – Browse & Search Vehicles	8
Table 4 UC-03 – Place Bid	9
Table 5 UC-04 – Verify User Account	9
Table 6 UC-05 – Manage Vehicle Listings	10
Table 7 UC-06 – Select Winning Bidder.....	10
Table 8 Technology Selection and Justification.....	13
Table 9 Current Progress of the Project	14

1 INTRODUCTION

1.1 Background

The international second-hand vehicle trade is a significant and growing sector, with Japan being one of the world's leading exporters of used vehicles. Japanese auto auctions are held weekly and attract thousands of vehicles across all segments. Exporters like Tapro Japan Co. Ltd. participate in these auctions, acquire vehicles, and resell them to international buyers in markets such as Sri Lanka, Kenya, and the Middle East.

Traditionally, communication between exporters and their international buyers has relied on informal channels: telephone calls, WhatsApp groups, and email threads. While workable at small scale, these approaches become increasingly unmanageable as the number of vehicles and buyers grows. The lack of a structured digital platform creates friction at every stage of the process, from listing vehicles to identifying the winning bidder.

1.2 Problem Statement

Tapro Japan Co. Ltd. currently lists vehicles and collects buyer interest through social media messaging platforms. The following specific problems have been identified through discussions with the client:

- Bids are placed informally and are difficult to track accurately, leading to disagreements over who holds the highest offer.
- There is no mechanism to restrict bidding to verified, trusted buyers; unverified parties may place bids without commitment.
- Vehicle information (images, specifications, auction dates, pricing) is communicated inconsistently across different messaging threads.
- The client has no centralized historical record of auctions, bids, or buyer activity, making trend analysis and business review impossible.
- As the client seeks to expand internationally, the manual process creates an operational bottleneck that prevents scaling.

1.3 Motivation

The proposed system directly addresses these problems by providing a dedicated, professional, web-based platform that centralizes the entire auction workflow. From an academic perspective, the project presents an opportunity to apply full-stack web development techniques to a real-world business problem involving multi-role access control, time-constrained transaction logic, and relational data modelling. The solution is expected to reduce manual workload for the client, improve buyer satisfaction through transparency, and provide a scalable foundation for future business growth.

1.4 Objectives

1. To develop a web-based platform that streamlines the client's vehicle auction process and improves transparency for international buyers.
2. To ensure secure and trustworthy participation by implementing proper user registration, account verification, and role-based access control.
3. To support efficient auction management by providing administrators with tools for vehicle management, auction scheduling, and winning-bidder selection.
4. To maintain accurate historical records and ensure system reliability, data integrity, and secure handling of all user interactions.
5. To refine and validate the system through stakeholder feedback and user acceptance testing aligned with real business needs.

2 ANALYSIS

2.1 Existing System

The current workflow at Tapro Japan Co. Ltd. can be broken down into three phases: vehicle listing, bid collection, and winner selection.

In the vehicle listing phase, the client photographs newly acquired vehicles and manually records specifications. These details are distributed to buyers through WhatsApp broadcast lists and Facebook posts. Each vehicle is presented individually in a message thread, with no consistent structure or standardisation.

Bid collection relies entirely on buyers replying to the relevant message thread with their offered price. The client manually reads through replies, identifies the current highest bid, and communicates this to interested parties. There is no real-time visibility for buyers into the current standing of their bid, nor any enforcement of minimum increments or bid validity periods.

Winner selection is performed manually by the client at the close of the auction period (typically when the vehicle must be committed to a shipping consignment). The client privately messages the winning bidder, and the transaction proceeds offline. This final step is opaque to other participants and can be perceived as unfair.

Key weaknesses of the existing system include: no audit trail, no access control, no structured data storage, dependence on the availability of the client to relay information, and poor scalability.

2.2 Literature Review

Online auction systems have been extensively studied in the literature and in practice. The design of such systems involves well-understood challenges spanning security, concurrency, and user experience. This section surveys the most relevant background to inform the design choices made in this project.

2.2.1 Online Auction Models

Klemperer [1] provides a comprehensive theoretical treatment of auction design, distinguishing between English (ascending-bid), Dutch (descending-bid), sealed-bid first-price, and Vickrey (second price) auctions. The client's current informal process most closely resembles an English auction, where buyers are aware of competing bids and can outbid one another until a deadline. This model has been retained in the proposed system as it is familiar to the client's buyers and maximizes revenue through competitive dynamics.

Wang et al. [2] analyze the trust mechanisms required in online auction marketplaces and conclude that verified identity and transparent bidding history are the two most critical factors in buyer confidence. Both are addressed explicitly in the functional requirements of this project.

2.2.2 Web Application Architecture

The three-tier architecture (presentation, application logic, and data layers) is the industry-standard pattern for web-based transaction systems [3]. Separation of concerns between the React.js frontend, the Node.js/Express backend API, and the PostgreSQL database ensures that each tier can be developed, tested, and scaled independently. REST (Representational State Transfer) is the dominant API design paradigm for modern web applications [5]. RESTful endpoints mapped to

HTTP verbs (GET, POST, PUT, DELETE) provide a stateless, scalable communication contract between the frontend and backend. This project adopts REST throughout the backend API design.

2.2.3 Authentication and Role Based Access Control

JSON Web Tokens (JWT) are a compact, URL-safe mechanism for representing claims between parties [6]. They are widely adopted for stateless authentication in single-page applications because they eliminate the need for server-side session storage. The proposed system uses JWT issued at login and validated on protected endpoints, with role claims (admin, verified_buyer, unverified_buyer) encoded in the token payload.

Role-Based Access Control (RBAC) is the standard model for enforcing privilege separation in multi-user systems [7]. The three-role model (admin, verified buyer, public) employed in this project follows the principle of least privilege: public users may only browse; unverified users may register; only admin-approved accounts may bid.

2.2.4 Concurrent Bid Handling

Race conditions in concurrent bid submission are a well-known problem in online auction systems [8]. If two buyers submit bids simultaneously, a naive implementation may accept both, potentially recording a lower bid as the winner. The proposed solution mitigates this using PostgreSQL's `SERIALIZABLE` transaction isolation level for bid insertion, ensuring that only one transaction succeeds when two attempt to modify the same auction record simultaneously.

2.2.5 Similar Commercial Systems

Platforms such as [Cars & Bids](#), [AutocomJapan](#) and [RamaDBK](#) provide reference implementations of web-based vehicle auction platforms. Analysis of these systems reveals common design patterns: paginated vehicle grids with rich filtering, countdown timers displayed to buyers, bid ladders showing historical offers, and admin dashboards for lot management. These patterns have been incorporated into the UI/UX design of this project.

2.3 Functional Requirements

The following table summarizes the functional requirements derived from client interviews and analysis of the existing workflow

Table 1 Functional Requirements classified by priority

ID	Requirement	Priority	Category
FR-01	User registration with email, password, and contact information	High	User Management
FR-02	Admin verification or rejection of registered user accounts	High	User Management
FR-03	Secure login with JWT-based authentication	High	User Management
FR-04	Role-based access control: public, verified buyer, admin	High	User Management
FR-05	Admin CRUD operations for vehicle listings (images, specs, auction date)	High	Vehicle Management
FR-06	Batch upload of weekly auction vehicles	Medium	Vehicle Management
FR-07	Public vehicle browsing with search and filter (make, model, year, price)	High	Vehicle Management
FR-08	Creation of auctions with configurable start/end datetime	High	Auction Management
FR-09	Automatic auction status transitions	High	Auction Management
FR-10	Verified users may place bids on active auctions	High	Bidding System
FR-11	Bid validation: minimum increment enforcement, auction timing check	High	Bidding System
FR-12	Real-time display of current highest bid to buyers	High	Bidding System
FR-13	Prevention of bidding after auction closure	High	Bidding System
FR-14	Complete bid history per auction, accessible to admin	High	Bidding System
FR-15	Admin dashboard: pending verifications, active auctions, bid summaries	High	Admin
FR-16	Admin selection of winning bidder per auction	High	Admin
FR-17	Email/in-app notifications for verification status and outbid events	Medium	Notifications
FR-18	Basic reporting: auction results, top buyers, bid counts	Medium	Reporting

2.4 Non-Functional Requirements

1. **Security:** Passwords stored using bcrypt hashing (minimum cost factor 10). All API endpoints protected by JWT validation middleware. SQL injection prevented through parameterized queries.
2. **Performance:**
 - API response time < 500 ms under normal load
 - System supports at least 100 concurrent users without performance degradation
 - Bid submission success rate $\geq 99\%$ during active auctions
3. **Reliability:** System available 99% of uptime during auction periods. Database transactions used to maintain data consistency during concurrent bid submission.
4. **Usability:** Responsive design supporting desktop and mobile browsers. Intuitive navigation requiring no training for returning buyers.
5. **Scalability:** Stateless backend API deployable behind a load balancer. PostgreSQL connection pooling via pg-pool to handle concurrent requests.
6. **Maintainability:** Codebase organized by feature modules. Inline documentation and a deployment guide provided to the client.

3 Design of Solution

3.1 System Architecture

The system follows a three-tier architecture comprising a React.js single-page application (SPA) at the presentation tier, a Node.js/Express REST API at the application tier, and a PostgreSQL relational database at the data tier. This separation ensures loose coupling between layers and allows each to evolve or scale independently.

The frontend communicates with the backend exclusively through HTTP REST calls. The backend enforces authentication via JWT middleware before permitting access to protected resources. The database layer is accessed through the node-postgres (pg) library, which provides connection pooling and parameterised query support. Figure 1 below describes this architecture conceptually.

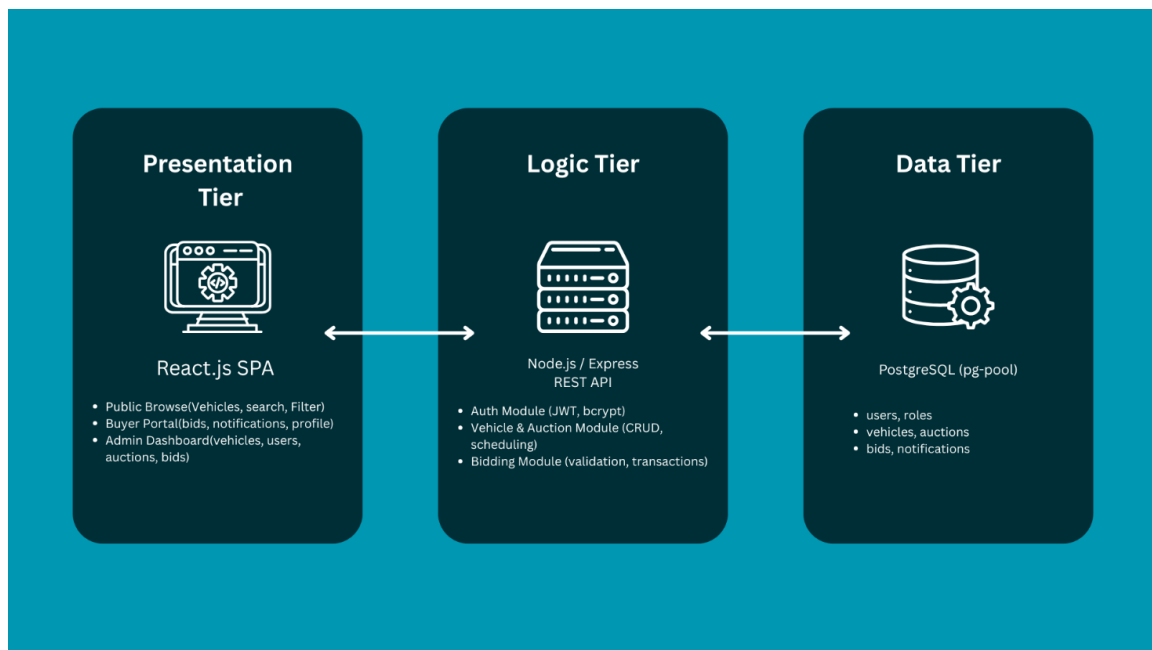


Figure 1 System High Level Architecture

3.2 Methodology

An Agile-inspired iterative development methodology has been adopted, with the project divided into four two-to-three-week sprints. Each sprint delivers a vertical slice of working functionality spanning frontend, backend, and database layers, rather than implementing a single architectural tier in isolation. This approach was chosen over a strict Waterfall model because requirements benefit from early user feedback on UI/UX before full implementation, and the bidding module carries technical risk that warrants early validation. Testing is integrated continuously throughout each sprint, with unit tests written alongside implementation and quantitative acceptance criteria applied at each sprint review: API endpoints verified against defined test cases, page load time under 3 seconds, and zero data-integrity failures under concurrent bid submission.

Sprint 1 (complete) delivered end-to-end user registration and login, covering database schema, authentication API, and login UI. Sprint 2 (in progress) delivers browsable vehicle listings across API, grid, and detail pages. Sprint 3 will implement the bidding module and UI as a complete working auction slice. Sprint 4 will cover the admin dashboard, winner selection, notifications, integration testing, user acceptance testing, and documentation finalisation.

3.3 Use Case Diagrams

Three primary actors interact with the system: Public User (unauthenticated), Verified Buyer (authenticated, admin-approved), and Administrator. The following use-case descriptions capture the core interactions.

Table 2 UC-01 – Register Account

Use Case	UC-01 – Register Account
Actor	Public User
Pre-condition	None
Main Flow	User navigates to the registration page, enters full name, email, password, phone number, and country. System validates inputs, hashes password, stores account with 'pending' status, and sends confirmation.
Post-condition	User account created with pending status; awaiting admin verification.

Table 3 UC-02 – Browse & Search Vehicles

Use Case	UC-02 – Browse & Search Vehicles
Actor	Public User / Verified Buyer
Pre-condition	System online
Main Flow	User visits vehicle listings page. Applies optional filters (make, model, year, price range). System returns paginated, filtered results with thumbnail images and key specs.
Post-condition	Filtered vehicle list displayed.

Table 4 UC-03 – Place Bid

Use Case	UC-03 – Place Bid
Actor	Verified Buyer
Pre-condition	User logged in and verified; auction is active
Main Flow	Buyer views vehicle detail page showing current highest bid. Enters bid amount. System validates: amount > current highest bid + minimum increment; auction not closed. If valid, bid is recorded in a serialisable transaction and the display refreshes.
Post-condition	New bid recorded; buyer's bid rank updated; previous high bidder notified.

Table 5 UC-04 – Verify User Account

Use Case	UC-04 – Verify User Account
Actor	Administrator
Pre-condition	Admin logged in
Main Flow	Admin views the pending-verification queue. Reviews user details. Clicks Approve or Reject. System updates account status. User is notified by email.
Post-condition	User status updated to 'verified' or 'rejected'.

Table 6 UC-05 – Manage Vehicle Listings

Use Case	UC-05 – Manage Vehicle Listings
Actor	Administrator
Pre-condition	Admin logged in
Main Flow	Admin accesses vehicle management panel. Creates a new listing by entering make, model, year, chassis number, mileage, grade, images, and starting price. Alternatively edits or deletes an existing listing.
Post-condition	Vehicle listing created/updated/deleted in the database.

Table 7 UC-06 – Select Winning Bidder

Use Case	UC-06 – Select Winning Bidder
Actor	Administrator
Pre-condition	Admin logged in; auction closed
Main Flow	Admin opens the closed auction detail page showing the full bid ladder. Admin clicks 'Select Winner' on the highest (or chosen) bid. System records the winner and triggers winner notification.
Post-condition	Winning bidder recorded; outcome visible on auction result page.

3.4 Entity Relationship Diagram

The core entities and their relationships are described below

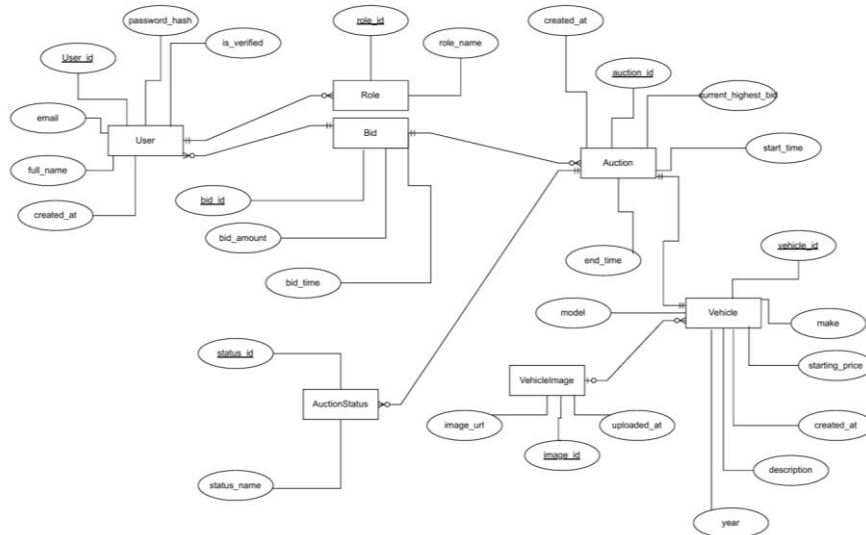


Figure 2 Entity Relationship Diagram

3.5 Logical Database Design

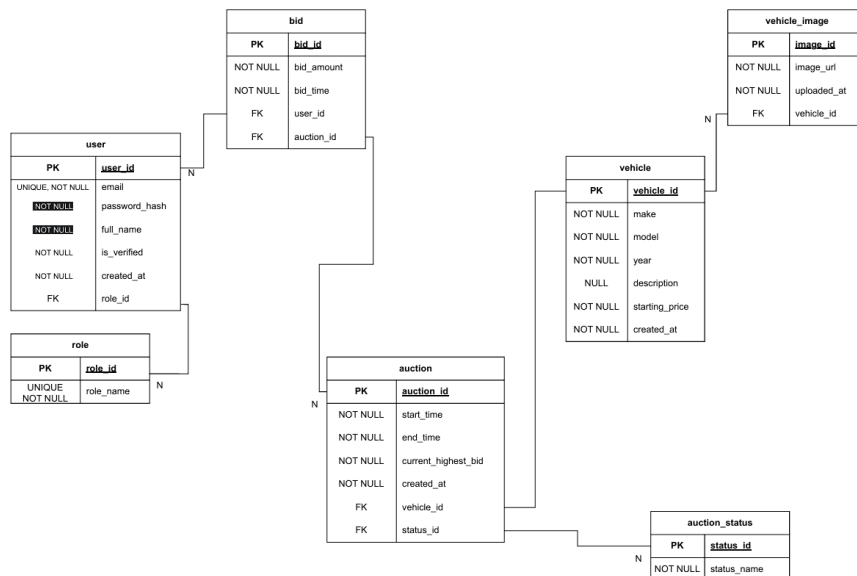


Figure 3 Logical Database Design

3.6 Class Diagram Design

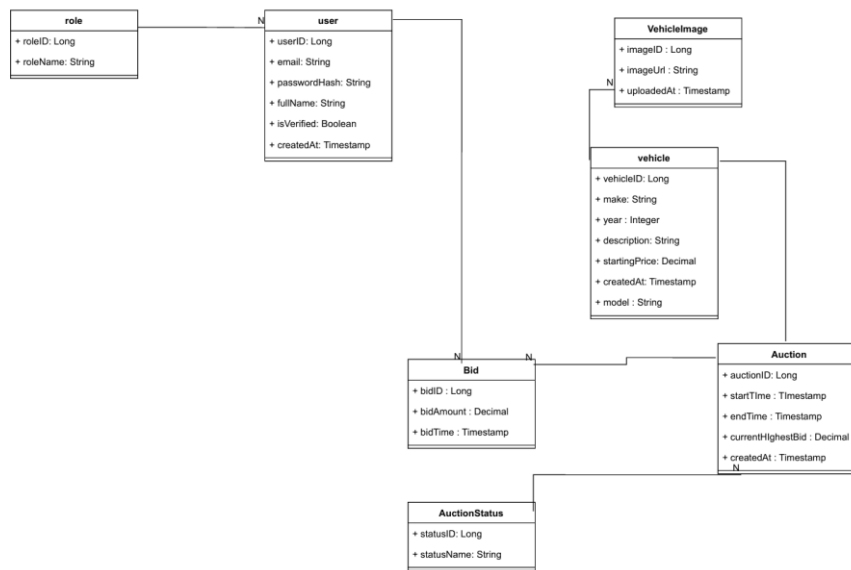


Figure 4 Class Diagram

3.7 Sequence Diagram Design

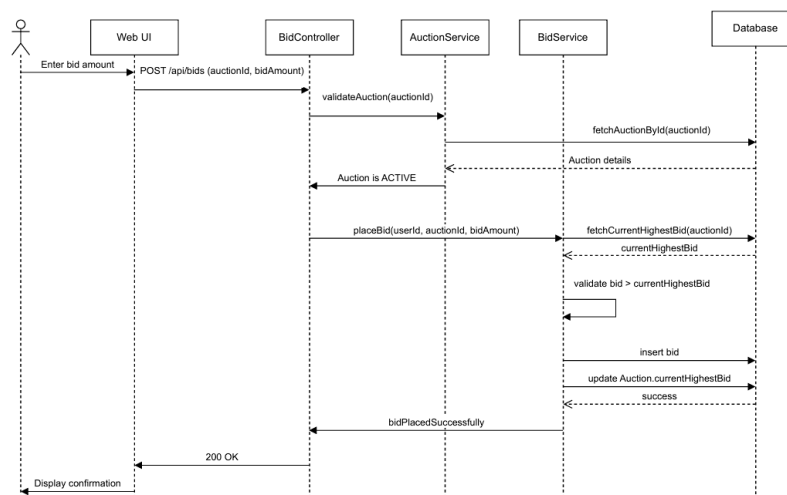


Figure 5 Sequence Diagram

3.8 Technology Selection

Multiple alternative technologies were evaluated before selecting the final stack. The table below summarizes the key decisions and their rationale.

Table 8 Technology Selection and Justification

Component	Technology	Justification
Frontend	React.js	Component-based SPA model well-suited to dynamic auction UI. Large ecosystem (React Router, Axios, TailwindCSS). Familiarity reduces learning curve.
Frontend alternative	Vue.js (rejected)	Comparable capability, but React's wider adoption provides better long-term maintainability for the client.
Backend	Node.js + Express	Non-blocking I/O is appropriate for concurrent bid submissions. JavaScript across frontend and backend reduces context-switching.
Backend alternative	Django REST (rejected)	More suitable for data-heavy applications; added Python dependency increases deployment complexity.
Database	PostgreSQL	ACID-compliant transactions are essential for bid integrity. SERIALIZABLE isolation prevents race conditions. Rich JSON support for vehicle images.
Database alternative	MySQL (rejected)	Weaker support for SERIALIZABLE isolation and array data types.
Authentication	JWT (jsonwebtoken)	Stateless; no server-side session store required. Claims-based roles simplify middleware logic.
Auth alternative	Session cookies (rejected)	Requires server-side session storage, complicating horizontal scaling.
Deployment	Node.js + Nginx (planned)	Nginx serves the React SPA and proxies API calls. Straightforward single-server deployment appropriate for the client's scale.

3.9 Current Development Status

The table below summarizes the current state of each development phase as of the submission date of this report (April 2026).

Table 9 Current Progress of the Project

Phase / Task	Target Date	Status	Notes
Requirements Analysis & Design	November 2025	Complete	SRS and design artefacts finalised.
Backend: Project scaffolding & DB setup	December 2025	Complete	Express app, pg-pool, migration scripts ready.
Backend: Authentication module	December 2025	Complete	Register, login, JWT middleware, role guard.
Backend: Vehicle management APIs	January 2026	Complete	CRUD endpoints for vehicles with image upload.
Frontend: Routing & layout scaffolding	February 2026	Complete	React Router, protected routes, base layout.
Frontend: Vehicle browsing & detail pages	March 2026	In Progress	Grid and filter UI built; detail page pending.
Backend: Auction management APIs	March 2026	In Progress	Create/list/update endpoints; status cron pending.
Backend: Bidding module	April 2026	Planned	Transaction-safe bid insertion, validation logic.
Frontend: Bidding UI & live updates	April 2026	Planned	Bid form, countdown timer, bid ladder.
Admin Dashboard (Frontend)	May 2026	Planned	Verification queue, auction controls, bid view.
Integration & UAT Testing	June 2026	Planned	End-to-end test suite; client UAT sessions.
Documentation & Final Report	June–July 2026	Planned	User manual, deployment guide, project report.

3.10 Design Evolution and Earlier Approaches

The design of the system did not arrive at its current form immediately. Several iterations of the security architecture, data model, and access control strategy were explored and refined during the analysis phase. This section documents the earlier approaches considered, the problems identified in each, and the rationale for the design decisions that were ultimately adopted. These artefacts serve as evidence of the iterative design process undertaken throughout the project.

3.10.1 Iteration 1: Initial Security and Authorization Design

The first formal design document produced during this project was a Security and Authorization Design specification. This document defined the authentication mechanism, authorization model, access control rules, and error handling policy for the platform. Key design decisions captured at this stage included:

Authentication via email and password, with passwords stored as hashed values using bcrypt and JWT issued on successful login for stateless session management.

A four-tier Role-Based Access Control (RBAC) model defining Guest (unauthenticated), Registered User (authenticated but unverified), Verified User (authenticated and admin-approved), and Admin roles.

Explicit access control rules mapping actions (view vehicles, view auction details, place bid, create auction, verify user, view bid history) to the minimum required role.

Verification enforcement at the backend application layer, with controller middleware checking verification status before granting access to bid-related service classes. Business rules related to security were to be consistently applied across all protected endpoints.

Standardised HTTP error responses: 401 Unauthorized for unauthenticated users and 403 Forbidden for insufficient permissions, with error responses designed not to expose internal system details.

3.10.2 Issues Identified and Design Refinements Made

Review of Iteration 1 against the functional requirements and data model revealed several issues that required revision before implementation could begin:

Boolean verification flag replaced with a three-state enum. The initial data model used a Boolean `is_verified` flag on the user entity. This is insufficient because it cannot represent the pending state—a newly registered user is neither verified nor rejected. The field was replaced with a `verification_status` enum (`'pending'`, `'verified'`, `'rejected'`), which correctly captures all three lifecycle states and enables the admin dashboard to surface a meaningful verification queue.

Role and AuctionStatus lookup tables replaced with inline enums. The initial ERD and class diagrams modelled Role and AuctionStatus as separate entities with their own primary keys and name attributes. While this follows a classic normalisation pattern, it introduces unnecessary joins on every authentication and auction status check for a fixed, non-dynamic set of values. Since neither roles nor auction statuses will be created or modified at runtime, they were replaced with

PostgreSQL ENUM types defined directly on the user and auction tables respectively, reducing query complexity without sacrificing data integrity.

- `current_highest_bid` redesigned as a derived value. The initial auction entity stored `current_highest_bid` as a mutable column updated on every bid insertion. This denormalised approach risks data inconsistency if a bid insertion succeeds but the subsequent update to `current_highest_bid` fails within the same transaction. In the revised schema, the current highest bid is derived at query time using `MAX(bid_amount)` over the bids table for the relevant auction, eliminating the redundancy. A cached column may be reintroduced as a performance optimisation at a later stage, but only with explicit documentation of the consistency contract.
- `winner_bid_id` and `min_bid_increment` added to the auction entity. The initial design omitted a mechanism for recording the admin's winning bidder selection (a core functional requirement) and the minimum bid increment (required by FR-11). Both fields were added to the auction entity: `winner_bid_id` as a nullable foreign key to the bid table, set by the admin after auction closure, and `min_bid_increment` as a numeric column with a default value configurable per auction.
- `model` attribute added to vehicle in the physical schema. The `model` attribute was present in both the ER diagram and class diagram but was omitted from the initial relational schema. This inconsistency was identified during cross-checking of the three design artefacts and corrected in the final physical schema.

These refinements illustrate that the design process was iterative and evidence-driven. The initial security specification provided a valuable baseline that made the gaps in the data model visible when the two artefacts were compared against the full set of functional requirements. The final design documented in Sections 3.4 and 3.5 reflects all of these corrections and represents the schema that is currently being implemented.

References

- [1] P. Klemperer, **Auctions: Theory and Practice**. Princeton, NJ, USA: Princeton University Press, 2004.
- [2] W. Wang, Z. Zhang, and H. Li, "Trust mechanisms in online auction markets: A review," **Electronic Commerce Research and Applications**, vol. 18, pp. 30–42, 2016.
- [3] M. Fowler, **Patterns of Enterprise Application Architecture**. Boston, MA, USA: Addison-Wesley, 2002.
- [4] L. Bass, P. Clements, and R. Kazman, **Software Architecture in Practice**, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2012.
- [5] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.
- [6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," **IEEE Computer**, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [8] M. P. Wellman, A. Greenwald, and P. Stone, **Autonomous Bidding Agents: Strategies and Lessons from the Trading Agent Competition**. Cambridge, MA, USA: MIT Press, 2007.